

Задача А. Новый автомобиль

Изначально сдвинем все точки так, чтобы центр колеса находился в начале координат $(0, 0)$. Будем двигать тормоз относительно центра колеса против часовой стрелки вместо всего колеса по часовой. Чем это лучше? Если мы начертим траекторию тормоза, то получится окружность с центром в начале координат и радиусом равным расстоянию от начала координат до тормоза.

Рассмотрим некоторое ребро с номером i . Выразим прямую из этого ребра и пересечём её с нашей окружностью. Найдём точку, которая принадлежит отрезку(ребру). Если такая точка есть, то это одна из точек пересечения окружности и многоугольника. Так повторим для всех ребёр.

Если мы не нашли точек пересечения многоугольника и окружности, то ответа нет. Иначе, если одна точка, то легко найти угол поворота многоугольника. Теперь мы имеем две точки и надо выбрать из них ту, которая ближе всего находится к тормозу на окружности, если идти от него против часовой стрелки. Эта точка будет иметь минимальную абсолютную разницу его полярного угла с полярным углом тормоза. Эта разница будет являться тем углом, на который нужно будет повернуть многоугольник по часовой стрелке.

Итоговая сложность решения зависит от его реализации, но мы допускали любое из них. $O(n)$ или $O(n \cdot \log(n))$

Задача В. Непесочные часы

Пусть прошло t секунд с начала суток. За это время секундная стрелка повернулась на $6 \cdot t$ градусов, минутная — на $\frac{1}{10} \cdot t$ градусов, часовая — на $\frac{1}{120} \cdot t$ градусов. Составим уравнения для углов между стрелками:

Секундная и минутная:

$$\frac{59}{10} \cdot t = 360 \cdot N \pm \frac{p}{q}$$

Секундная и часовая:

$$\frac{719}{120} \cdot t = 360 \cdot N \pm \frac{p}{q}$$

Минутная и часовая:

$$\frac{11}{120} \cdot t = 360 \cdot N \pm \frac{p}{q}$$

Здесь N — любое целое неотрицательное число. Ясно, что N не может быть больше, чем секундная стрелка сделает оборотов за 12 часов (а это $60 \cdot 12 = 720$). Тогда для любой пары стрелок можно перебрать N и вычислить все ответы, проверив, что время входит в требуемый интервал.

Задача С. Стеклотара

В задаче вам были даны 3-д изображения цифр, составленных из кубиков. Двухцветными назывались кубики с ровно 4-я видимыми гранями (на самом деле 2 невидимые грани грани у них были бесцветные, а 4 видимые нужны было покрасить в одинаковый для всех цифр цвет). На вход подавалось очень длинное число n , составленное из этих цифр. Также были даны 3 целых числа R , G и B , они обозначали количество стекляшек разных цветов. Нужно было определить, хватит ли стекляшек каждого цвета на покраску всего числа n (точнее на покраску всех двухцветных кубиков цифр этого числа).

Задача заключалась в том, чтобы посчитать количество двухцветных кубиков в каждой цифре и занести их в массив руками, а затем считать итоговое количество таких кубиков в заданном числе n , домножая его на 4, мы получаем количество стекляшек, необходимое на покраску числа. Далее нужно просто проверить какие из чисел R , G , B больше либо равны нужному числу стекляшек (и, конечно, не забыть вывести «No», если ни одно из чисел не подходит).

Задача D. Разности

Напомним условие задачи: вам дан массив из n целых неотрицательных чисел, вам нужно выбрать некоторые k индексов $j_1, j_2, \dots, j_k$ таким образом, чтобы значение $\sum_{i=2}^k |a_{j_{i-1}} - a_{j_i}|$ было максимальным.

Итак, давайте заметим следующий факт: если мы будем раскрывать модули, т.е. допустим у нас есть $|a - b|$ и при этом $a < b$, тогда если мы случайно перепутаем знаки, т.е. поставим $+a - b$ вместо $-a + b$, то ответ станет меньше.

Теперь напомним следующее ДП, которое будем пересчитывать по слоям: на j -м слое, dp_i - максимальный ответ, когда мы уже выбрали j индексов и последний выбранный индекс был равен i . Тогда давайте научимся пересчитывать это ДП за линию. Пусть $mx1$ и $mx2$ изначально равны $-\infty$, тогда будем идти слева направо по массиву и строить массив new_dp , как $new_dp_i = \max(mx1 - a_i, mx2 + a_i)$, и обновлять $mx1$ и $mx2$, как $mx1 = \max(mx1, dp_i + a_i)$, $mx2 = \max(mx2, dp_i - a_i)$. И благодаря замечанию, описанному в предыдущем абзаце, мы точно посчитаем ответ.

Задача Е. Покупка автомобиля

Выпишем произведение чисел массива a_i и массива b_i :

$$a_1 \cdot a_2 \cdot \dots \cdot a_n = A$$

$$b_1 \cdot b_2 \cdot \dots \cdot b_n = B$$

Давайте прологарифмируем обе стороны каждого уравнения. Получаем:

$$\log(a_1) + \log(a_2) + \dots + \log(a_n) = \log(A) = A'$$

$$\log(b_1) + \log(b_2) + \dots + \log(b_n) = \log(B) = B'$$

Легко понять, что A' и B' получатся небольшие, поэтому никакого переполнения, в отличие от прямой постановки задачи, мы не получим. Тогда давайте переберём индекс i , элемент которого мы заменим. Тогда давайте напишем бинарный поиск, который найдет минимальное число x такое, что сумма всех логарифмов без i -го элемента $+\log(x)$ была не меньше B' . Если $x \leq a_i$, то к ответу прибавим $m - x$, потому что мы не можем заменить a_i на саму себя по условию, иначе $m - x - 1$.

Задача F. Холодильник

Опять же, напомним условие задачи: у Вас есть массив a и последовательность Фибоначчи fib , Вам приходят q запросов двух типов, 1 - добавить ко всем элементам массива, с индексами от l до r , целое положительное число x , 2 - узнать сумму $\sum_{i=1}^{r-l+1} fib_i \cdot a_{l+i-1}$ по модулю $10^9 + 7$.

Построим дерево отрезков, в котором для каждой вершины v будем хранить следующие величины: $t1_v = fib_i \cdot a_{l+i-1}$ и $t2_v = fib_{i+1} \cdot a_{l+i-1}$, где $i = 1 \dots r - l + 1$, а l и r - границы отрезка в вершине ДО. Тогда предсчитаем последовательность fib тривиальным способом и насчитаем над ней префиксные суммы $pref$. **Обратите внимание, все переменные следует считать по модулю, чтобы избежать переполнения.**

Теперь запрос обновления выполняется тривиально: для всех вершин, перескающих с отрезком запроса, мы обновляем $t1_v$ и $t2_v$, используя префиксные суммы, а для вершин, полностью лежащих в запросе, мы также сохраняем $p_v + = x$ (проталкивания нужно писать аккуратно, чтобы случайно не словить TLE).

Осталось научиться считать ответ. Главный вопрос в том, как из наших значений, которые считают суммы начиная с fib_1 и fib_2 , перейти к суммам с fib_x . На самом деле, сделать это довольно просто: рассмотрим последовательность фибоначчи, которая начинается с fib_x, fib_{x+1} , тогда последующая последовательность принимает следующий вид - $fib_x + fib_{x+1}, fib_x + 2 \cdot fib_{x+1}, 2 \cdot fib_x + 3 \cdot fib_{x+1}, 3 \cdot fib_x + 5 \cdot fib_{x+1}, \dots$. Я думаю, Вы уже заметили тут еще одну последовательность Фибоначчи. Теперь научимся явно считать ответ для вершины, полностью лежащей в запросе. Итак, с ранее описанными фактами, легко вывести следующие случаи (пусть границы запроса - cl и cr , а границы вершины в ДО - l и r):

1. Если $l == cl$, возвращаем $t1_v$.
2. Если $l == cl + 1$, возвращаем $t2_v$.
3. Иначе, возвращаем $fib_{l-cl} * t2_v + fib_{l-cl-1} * t1_v$.

Задача G. Уязвимая служба доставки

Формальное условие задачи было следующим:

У вас есть неориентированный граф и q запросов включить или выключить вершину v . После каждого запроса не менее 1 и не более 10 включенных вершин. После каждого запроса нужно

было вывести количество способов выбрать некоторое подмножество уязвимых вершин. Что такое **уязвимая** вершина? Это такая вершина, что она при её удалении из графа нельзя будет достичь **некоторой** включенной вершины.

Данную задачу можно было решать двумя способами, но опишу решение одного из них - авторского.

Давайте научимся решать задачу для дерева. Понятно, что для уязвимые вершины для некоторого множества включенных вершин будут все вершины, которые находятся на некотором пути от 1 до включенной вершины.

После каждого запроса сожмём наше дерево. Каким образом? В дереве останутся все наименьшие общие предки каждой пары включенных вершин, а также корень дерева и каждая включенная вершина. Граф можно построить следующим способом:

Отсортируем все вершины сжатого графа по времени выхода её из обхода в глубину. Этот порядок будет являться топологической сортировкой дерева. Будем поддерживать множество вершин, которым мы не выставили предков, в порядке возрастания времени выхода. Когда встречаем вершину, то будем убирать с начала нашего множества вершины, у которых время выхода меньше, чем у текущей вершины. Те вершины, которые удаляем, являются непосредственными сыновьями текущей вершины. После удаления и обновления предков добавим в множество текущую вершину и продолжим действие дальше на следующей вершине из топсорта.

В этом графе для каждой вершины будем хранить количество вершин, включая саму вершину, сверху до ближайшего предка, который входит в сжатый граф. Назовём эту характеристику вершины len_v . Введём ДП f_v - количество способов выбрать некоторое подмножество поддерева v , а так же его len_v предков, в изначальном дереве так, что нельзя достичь всех включенных вершин из этого поддерева. Давайте обрабатывать вершины в порядке топсорта (то же самое, что и просто dfs написать). Допустим мы находимся в вершине v . Если эта вершина является включенной, то количество способов будет для него $f_v = (2^{len_v} - 1) \cdot 2^{siz_v}$, где siz_v — сумма всех len_u в его поддерева, не включая саму вершину v . Почему так? Потому что нам нельзя достигать саму вершину v , следовательно нас устраивают все подмножества самой вершины и предков, количество которых len_v , кроме подмножества, где все вершины не удалены. Так как мы не достигаем вершину v , то мы никогда не достигнем все вершины из его поддерева, следовательно мы можем выбрать любое подмножество из поддерева. Теперь случай, если у нас вершина v не включена. Тогда нам надо покрыть все включенные вершины из поддерева. Тогда нас удовлетворяют все случаи, когда все поддеревья непосредственных сыновей v уже покрыты и мы не удаляем ни одну вершину из len_v . Количество таких способов равно произведению всех f_u непосредственных сыновей u вершины v . Теперь случай, когда мы включаем хотя бы одну вершину из len_v . Мы таким образом покрываем все включенные вершины поддерева. Этот случай аналогичен случаю, когда v включена, так как случай с пустым множеством удаленных вершин из len_v уже обработали и мы хотим включить хотя бы одну вершину из этого множества. Итого, для случая, когда v не включена, $f_v = \prod f_u + (2^{len_v} - 1) \cdot 2^{siz_v}$. Итого ответ для запроса будет находится в f_1 .

Как прийти к решению задачи на дереве? Можно заметить, что определение уязвимой вершины подходит под определение доминатора одной из включенных вершин. Тогда давайте построим дерево доминаторов на данном графе и решим задачу на дереве.

Итого решение выходит за $O(q \cdot on^2 \cdot \log(n))$, где on — максимальное количество включенных вершин после каждого запроса.

Про построение дерева доминаторов можно почитать [wiki-algocode](#).

Задача Н. Тот ещё шахматист

Пусть мы сделали a, b, c, d ходов каждого из направлений соответственно и попали клетку (n, m) . Тогда, посчитав координаты по осям Ox и Oy , получим систему:

$$\begin{cases} 2a + 2b + c - d = n \\ -a + b + 2c + 2d = m \end{cases}$$

Сложив их, видим, что $a + 3b + 3c + d = n + m$, следовательно $3b + 3c \leq n + m$. Переберём все подходящие под это неравенство пары (b, c) и высчитаем из системы a и d . Если a и d оказались

целыми неотрицательными, то для такой четвёрки (a, b, c, d) существует $\frac{(a+b+c+d)!}{a!b!c!d!}$ способов составить уникальный путь, которые мы добавим к ответу. Факториалы и обратные факториалы по модулю можно предпросчитать.

Сложность алгоритма - $O(n^2)$

Задача I. Добыть улики

Согласно гипотезе Пака о системе локальных ходов (доказательство: <https://www.turgor.ru/1ktg/2009/4/articles/tetro.pdf>) замостить можно только прямоугольники, длины сторон которых кратны 4.

Задача J. Сдвиги

При вставке числа в матрицу меняется характеристика не всех элементов, а только:

- Всех элементов ~~первого~~ нулевого столбца (у каждого из них появляется левый сосед)
- Всех элементов последнего столбца (они переходят в первый столбец, то есть они все теряют левого соседа и получают правого)
- Элемент $a_{0,0}$ после вставки перейдёт в позицию $(0, 1)$ и получит нового левого соседа
- Элемент $a_{0,m-1}$ после вставки перейдёт в позицию $(1, 0)$ и получит нового соседа сверху

Заметив это, можно препросчитать для каждого столбца 3 суммарных характеристики:

- Полную
- Если представить, что у каждого элемента столбца нет левого соседа
- Если представить, что у каждого элемента столбца нет правого соседа

Сделав такой препросчёт можно за $O(1)$ обновлять ответ после каждого запроса по правилам, описанным выше.